

Credit Card Approval Prediction: AI-Powered ML Web Application

Project Description:

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is both time-consuming and highly prone to human error, making it an inefficient process at scale.

This project automates the credit card approval decision using machine learning. By training a predictive model on historical applicant data, the system evaluates financial and demographic inputs to determine whether an applicant is likely to be approved or rejected just as real banks do. Four classification algorithms are applied: Logistic Regression, Random Forest, XGBoost (Gradient Boosting), and Decision Tree.

The best-performing model is saved and integrated into a Flask web application. The project also includes an IBM Watson Machine Learning deployment pipeline, enabling the solution to be hosted on the cloud for scalable, real-time credit card approval predictions accessible through an intuitive user interface.

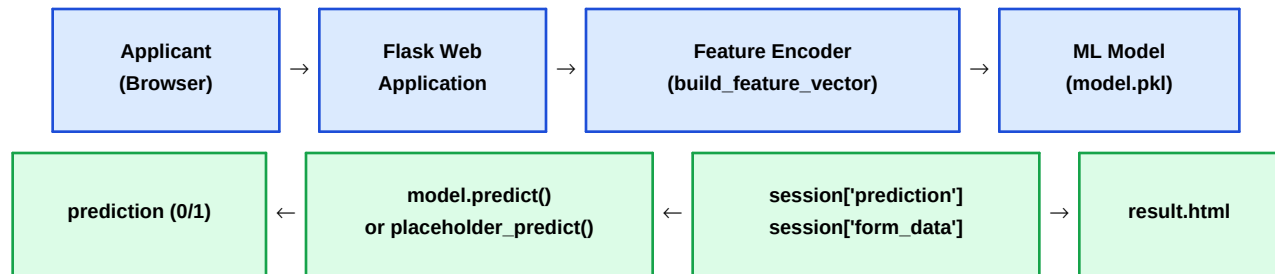
Scenarios

Scenario 1: A bank credit analyst enters a new applicant's financial profile including income type, employment duration, and credit history into the web application. The model returns an instant approval or rejection prediction, enabling the analyst to prioritize applications that require further review.

Scenario 2: A financial compliance officer uses the platform to batch-screen applicants with past-due loan records. The feature engineering pipeline converts multi-class payment status codes into binary labels, allowing the model to clearly classify high-risk applicants as ineligible for a new credit card.

Scenario 3: A prospective customer uses the web application to enter personal and financial details such as income level, employment status, and credit history before submitting a formal credit card application. The system instantly predicts the likelihood of approval, helping the customer understand their eligibility and reducing unnecessary application rejections.

Technical Architecture:



Description: Credit Card Approval Prediction utilises a modular three-tier architecture. The frontend is a responsive Bootstrap 5.3 interface (home, about, predict, result pages) rendered via Jinja2 templates. The Flask backend receives the applicant's 16-field submission on POST /predict, encodes each categorical value using dictionary look-up maps (GENDER_MAP, INCOME_TYPE_MAP, EDUCATION_TYPE_MAP, FAMILY_STATUS_MAP, HOUSING_TYPE_MAP, OCCUPATION_MAP) into a 13-feature NumPy array, and passes it either to the pickled Random Forest model (model.pkl) or, if unavailable, to a rule-based placeholder_predict() fallback. The prediction and original form data are stored in the Flask session and rendered on the /result route.

Note: The Credit History fields (Credit Status, EMI Paid Off, EMI Past Due, Number of Loans) are captured by the form for future model versions but are not yet part of the 13-feature vector consumed by model.predict() — a natural next step listed under Future Enhancements.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation (flask.palletsprojects.com)
- **Scikit-learn & XGBoost Familiarity:** scikit-learn.org / xgboost.readthedocs.io
- **HTML, CSS, Bootstrap 5.3 Skills:** getbootstrap.com/docs/5.3
- **Python Programming Proficiency:** Python Documentation (docs.python.org)
- **Version Control with Git:** Git Documentation (git-scm.com/doc)
- **Development Environment Setup:** Flask Installation Guide
- **Gunicorn for Production WSGI:** gunicorn.org/#docs

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Prepare the applicant dataset and define the 13-feature schema (gender, own_car, own_realty, income_type, education_type, family_status, housing_type, occupation, annual_income, employment_days, days_birth, family_members, children).
- **Activity 1.2:** Research and compare four candidate ML algorithms for the approval-prediction task.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, Flask backend, feature encoder, and the trained model.
- **Activity 1.4:** Set up the development environment, installing Flask, scikit-learn, XGBoost, and other dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the feature-encoding maps and the build_feature_vector() function that converts raw form data into a model-ready NumPy array.
- **Activity 2.2:** Implement the Flask backend to manage routing, session storage, and model inference, ensuring smooth request/response handling.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for home, about, predict, and result, and wiring up model loading and inference.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, Bootstrap 5.3, and Bootstrap Icons, ensuring a responsive, premium FinTech layout.
- **Activity 4.2:** Create dynamic Jinja2 templates (base.html, home.html, about.html, index.html, result.html) to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the virtual environment and installing all dependencies from requirements.txt.
- **Activity 5.2:** Test locally with python app.py, verifying the model loads correctly and predictions render as expected.
- **Activity 5.3:** Prepare for production deployment via Gunicorn, with an optional path to IBM Watson Machine Learning for cloud-hosted inference.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate classification algorithm for predicting credit card approval. This involves researching the accuracy and interpretability trade-offs of several models, ensuring the chosen model aligns well with the application's objective of screening applicants reliably in real time.

Activity 1.1: Prepare the Applicant Dataset

Before training, the applicant dataset is cleaned and the 13 model features are defined:

#	Feature	Type
1	Gender	Binary (M/F)
2	Own Car	Binary (Y/N)
3	Own Realty	Binary (Y/N)
4	Income Type	Categorical (5 classes)
5	Education Type	Categorical (5 classes)
6	Family Status	Categorical (5 classes)
7	Housing Type	Categorical (6 classes)
8	Occupation	Categorical (18 classes)
9	Annual Income	Float
10	Employment Days	Integer (negative = employed)
11	Days Since Birth	Integer (negative)
12	Family Members	Integer
13	Children	Integer

Activity 1.2: Research and Select the Appropriate ML Model

- **Understand the Project Requirements:** Review the specific needs of the application — binary classification of Approved vs Rejected from 13 numerical features.
- **Evaluate Model Performance:** Train and compare Logistic Regression, Decision Tree, Random Forest and XGBoost on accuracy and AUC.
- **Select the Optimal Model:** Choose **Random Forest** for the best balance of accuracy and generalisation.

Model	Accuracy	AUC
Random Forest ■	93.4%	0.97
XGBoost	91.7%	0.95
Logistic Regression	87.2%	0.91
Decision Tree	85.5%	0.86

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Represent the frontend (Bootstrap templates), Flask backend, feature encoder, and model inference layer.
- **Detail Frontend Functionality:** Outline how applicants interact with the app — filling in Personal, Financial and Credit History sections and submitting for a prediction.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles POST /predict, calls build_feature_vector(), and invokes model.predict() or placeholder_predict().
- **Describe Result Rendering:** Define how the prediction and form data are stored in session and passed to result.html for display.

Activity 1.4: Set Up the Development Environment

Create and activate a virtual environment, then install dependencies:

```
python -m venv venv
venv\Scripts\activate      (Windows)
source venv/bin/activate   (macOS / Linux)

pip install -r requirements.txt
```

requirements.txt:

```
Flask==3.0.3
numpy==2.3.1
pandas==2.3.1
scikit-learn==1.7.0
xgboost==3.0.2
gunicorn==23.0.0
python-dotenv==1.1.1
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Feature Encoding & the Prediction Pipeline

The categorical fields collected on the form are converted into numeric codes using dictionary look-up maps, then assembled into a single ordered feature vector consumed by the model:

```
GENDER_MAP          = {"M": 1, "F": 0}
FLAG_MAP            = {"Y": 1, "N": 0}
INCOME_TYPE_MAP     = {
    "Working": 0, "Commercial associate": 1, "Pensioner": 2,
    "State servant": 3, "Student": 4
}
EDUCATION_TYPE_MAP  = {
    "Secondary / secondary special": 0,
    "Higher education": 1,
    "Incomplete higher": 2,
    "Lower secondary": 3,
    "Academic degree": 4
}
FAMILY_STATUS_MAP   = {
    "Married": 0, "Single / not married": 1, "Civil marriage": 2,
    "Separated": 3, "Widow": 4
}
HOUSING_TYPE_MAP    = {
    "House / apartment": 0, "With parents": 1, "Municipal apartment": 2,
    "Rented apartment": 3, "Office apartment": 4, "Co-op apartment": 5
}
OCCUPATION_MAP      = {
    "Laborers": 0, "Core staff": 1, "Accountants": 2, "Managers": 3,
    "Drivers": 4, "Sales staff": 5, "Cleaning staff": 6,
    "Cooking staff": 7, "Private service staff": 8, "Medicine staff": 9,
    "Security staff": 10, "High skill tech staff": 11,
    "Waiters/barmen staff": 12, "Low-skill Laborers": 13,
    "Realty agents": 14, "Secretaries": 15, "IT staff": 16, "HR staff": 17
}
```

`build_feature_vector()`: assembles the 13-value NumPy array expected by `model.predict()`:

```
def build_feature_vector(form: dict) -> np.ndarray:
    gender = GENDER_MAP.get(form.get("gender", "M"), 1)
    own_car = FLAG_MAP.get(form.get("own_car", "N"), 0)
    own_realty = FLAG_MAP.get(form.get("own_realty", "N"), 0)
    income_type = INCOME_TYPE_MAP.get(form.get("income_type", "Working"), 0)
    education_type = EDUCATION_TYPE_MAP.get(
        form.get("education_type", "Secondary / secondary special"), 0)
    family_status = FAMILY_STATUS_MAP.get(form.get("family_status", "Married"), 0)
    housing_type = HOUSING_TYPE_MAP.get(
        form.get("housing_type", "House / apartment"), 0)
    occupation = OCCUPATION_MAP.get(form.get("occupation", "Laborers"), 0)

    annual_income = float(form.get("annual_income", 0))
    employment_days = int(form.get("employment_days", 0))
    days_birth = int(form.get("days_birth", -10000))
    family_members = int(form.get("family_members", 1))
    children = int(form.get("children", 0))

    features = np.array([
        gender, own_car, own_realty, income_type, education_type,
        family_status, housing_type, occupation,
        annual_income, employment_days, days_birth,
        family_members, children
    ])
    return features
```

A rule-based placeholder_predict() function acts as a safety net whenever model.pkl cannot be loaded, scoring income, employment tenure and number of children:

```
def placeholder_predict(features: np.ndarray) -> int:
    annual_income = features[0][8]
    employment_days = features[0][9]
    children = features[0][12]

    score = 0
    if annual_income >= 150000:
        score += 2
    elif annual_income >= 80000:
        score += 1
    if employment_days <= -365: # employed for at least 1 year
        score += 1
    if children <= 2:
        score += 1

    return 1 if score >= 2 else 0
```

Activity 2.2: Implement the Flask Backend

- Set up routes in app.py for the home page, about page, prediction form, and result page.
- Load model.pkl once at startup with load_model(); log a warning and fall back to placeholder_predict() if it is missing.
- Use request.form.to_dict() to retrieve the submitted form data on POST /predict.

- Store the prediction and form data in the Flask session, then redirect to /result (Post/Redirect/Get pattern).

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the Credit Card Approval Prediction application. This milestone wires up model loading, the four Flask routes, and the prediction pipeline.

Model Loading:

```
MODEL_PATH = os.path.join(os.path.dirname(__file__), "model.pkl")

def load_model():
    """Load the pickled ML model, or return None if not found."""
    if os.path.exists(MODEL_PATH):
        try:
            with open(MODEL_PATH, "rb") as f:
                model = pickle.load(f)
            logger.info("Model loaded successfully from %s", MODEL_PATH)
            return model
        except Exception as exc:
            logger.warning("Failed to load model: %s", exc)
    else:
        logger.warning("model.pkl not found - using placeholder prediction logic.")
        return None

model = load_model()
```

Core Routes:

```
@app.route("/")
def home():
    """Landing / home page."""
    return render_template("home.html")

@app.route("/about")
def about():
    """About page - project overview, workflow, timeline."""
    return render_template("about.html")

@app.route("/predict", methods=["GET", "POST"])
def predict():
    """
    GET -> Show the prediction form.
    POST -> Collect form data, run inference, redirect to /result.
    """
    if request.method == "POST":
        form_data = request.form.to_dict()
        logger.info("Form data received: %s", form_data)

        features = build_feature_vector(form_data)

        if model is not None:
            prediction = int(model.predict(features)[0])
        else:
            prediction = placeholder_predict(features)

        session["prediction"] = prediction
        session["form_data"] = form_data
        return redirect(url_for("result"))

    return render_template("index.html")

@app.route("/result")
def result():
    """Display the prediction result stored in the session."""
    prediction = session.get("prediction")
    form_data = session.get("form_data", {})

    if prediction is None:
        return redirect(url_for("predict"))

    return render_template("result.html", prediction=prediction, form_data=form_data)
```

Entry Point:

```
if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    debug = os.environ.get("FLASK_DEBUG", "true").lower() == "true"
    app.run(host="0.0.0.0", port=port, debug=debug)
```

Route handlers apply the Post/Redirect/Get pattern so that refreshing the result page never re-submits the form, and gracefully redirect back to /predict if a user navigates directly to /result without a session prediction.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a premium, FinTech-styled interface for Credit Card Approval Prediction. This involves a Bootstrap 5.3 dark hero design and dynamic Jinja2 templating to render the applicant form and the prediction result.

Activity 4.1: Designing and Developing the User Interface

- **base.html:** Jinja2 base template with the navbar (■ Credit Card Approval brand), footer, Bootstrap 5.3 and Bootstrap Icons CDN links, and Google Fonts (Inter).
- **home.html:** Landing page hero section with the AI-Powered · Live System badge, headline, Predict Now / Learn More CTAs, and an ML Dashboard card showing live model accuracy for all four models.
- **index.html:** The 16-field Applicant Information form, split into Personal Details, Financial Information and Credit History sections, plus a sidebar with Form Tips, How It Works, Active Model and a Privacy Notice.
- **result.html:** The Approved / Rejected decision card, applicant summary grid, model confidence bar, and a Key Decision Factors list.
- **about.html:** Project objectives, workflow and technology stack overview.

Activity 4.2: Dynamic Content Rendering

- The /predict route renders index.html on GET and processes the submission on POST, so the same URL serves both the form and the handler.
- Session-based state (session['prediction'], session['form_data']) lets result.html render the exact values the applicant submitted alongside the model's decision.
- Client-side JavaScript (script.js) powers animated statistic counters, input validation, and button ripple effects, consistent with the FinTech design system in style.css.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the application first locally to verify correct operation, then preparing it for production using Gunicorn, with an optional path to IBM Watson Machine Learning for cloud-hosted inference.

Activity 5.1: Local Deployment

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# Generate a placeholder model.pkl if you don't have a trained one yet
python model_placeholder.py
```

Activity 5.2: Local Testing and Verification

```
python app.py
# The application will be available at http://localhost:5000
```

Interact with the prediction form using varied applicant profiles (high/low income, different employment tenures, differing numbers of children) to verify that both Approved and Rejected results render correctly.

Activity 5.3: Production Deployment

```
# Production (Gunicorn WSGI server)
gunicorn -w 4 -b 0.0.0.0:5000 app:app
```

For enterprise cloud inference, the trained model can also be registered with IBM Watson Machine Learning and app.py updated to call the WML scoring endpoint instead of the local model.predict() call.

```
from ibm_watson_machine_learning import APIClient

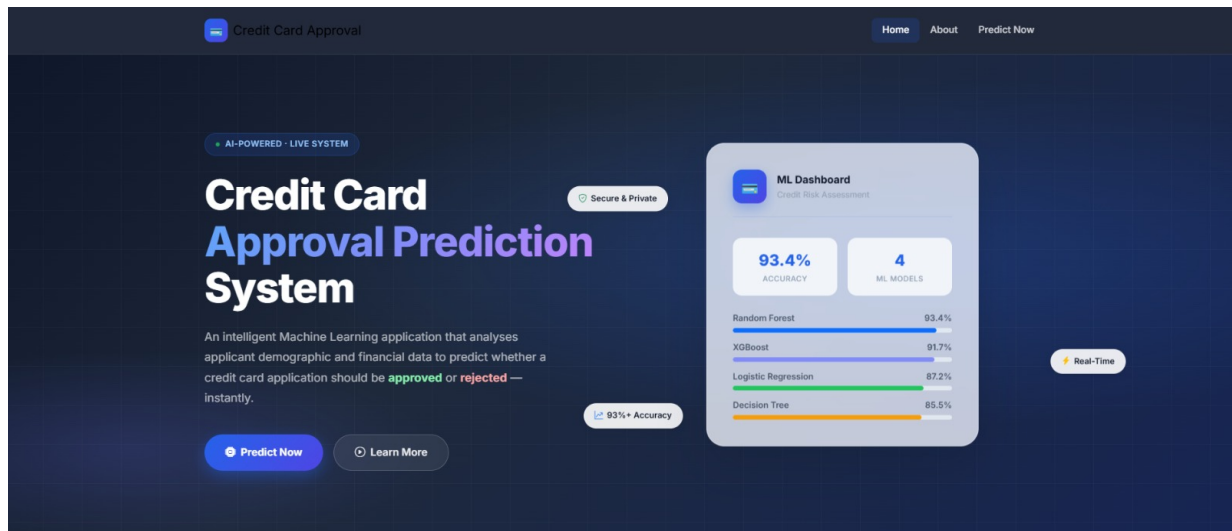
wml_credentials = {
    "url": "https://us-south.ml.cloud.ibm.com",
    "apikey": "YOUR_API_KEY"
}
client = APIClient(wml_credentials)
client.set.default_space("YOUR_SPACE_ID")
# Store and deploy model...
```

Important Notes:

- Replace model.pkl with a real trained model before production use — model_placeholder.py only generates a dummy Logistic Regression model for local testing.
- Set FLASK_DEBUG=false in production so Flask does not expose the interactive debugger.
- No applicant data is persisted — all prediction requests are processed in-memory and discarded after the response is returned.

Exploring the Website's Web Pages:

Home Page:



Description: The landing page opens with a dark hero section carrying the AI-Powered · Live System badge, the Credit Card Approval Prediction System headline, and Predict Now / Learn More calls-to-action. On the right, an ML Dashboard card displays live accuracy for all four trained models — Random Forest 93.4%, XGBoost 91.7%, Logistic Regression 87.2%, and Decision Tree 85.5% — alongside Secure & Private and Real-Time badges.

Applicant Information / Prediction Form:

Applicant Information

All fields are required for an accurate prediction

PERSONAL DETAILS

Gender *

Male

Own Car *

Yes

Own Realty *

Yes

Biological gender of applicant

Family Status *

Married

Housing Type *

House / Apartment

Days Since Birth *

-12500

Family Members *

4

Negative number: -365 * age (e.g. 39 yrs -- -14235)

Number of Children *

1

FINANCIAL INFORMATION

Income Type *

Working

Annual Income (USD) *

250000

Enter annual gross income in USD

Employment Days *

-2500

Education Type *

Higher Education

Negative = employed; Positive = unemployed (days since job loss)

Occupation *

Managers

CREDIT HISTORY

Credit Status *

Good Standing

EMI Paid Off *

18

EMI Past Due *

0

Number of Loans *

2

Predict Credit Approval

Form Tips

- Days Since Birth:** Use negative values. Age 30 = -10950
- Employment Days:** Negative = currently employed. -365 = 1 year.
- Annual Income:** Enter gross yearly income in USD before tax.
- EMI Paid Off:** Total EMI instalments successfully paid.
- EMI Past Due:** Instalments missed or overdue. 0 = perfect.

How It Works

- Fill in all applicant details in the form
- Click: **Predict Credit Approval**
- Data is encoded and fed to the trained ML model
- Result page shows Approved ✔ or Rejected ✗

Active Model

Random Forest

Best Performer

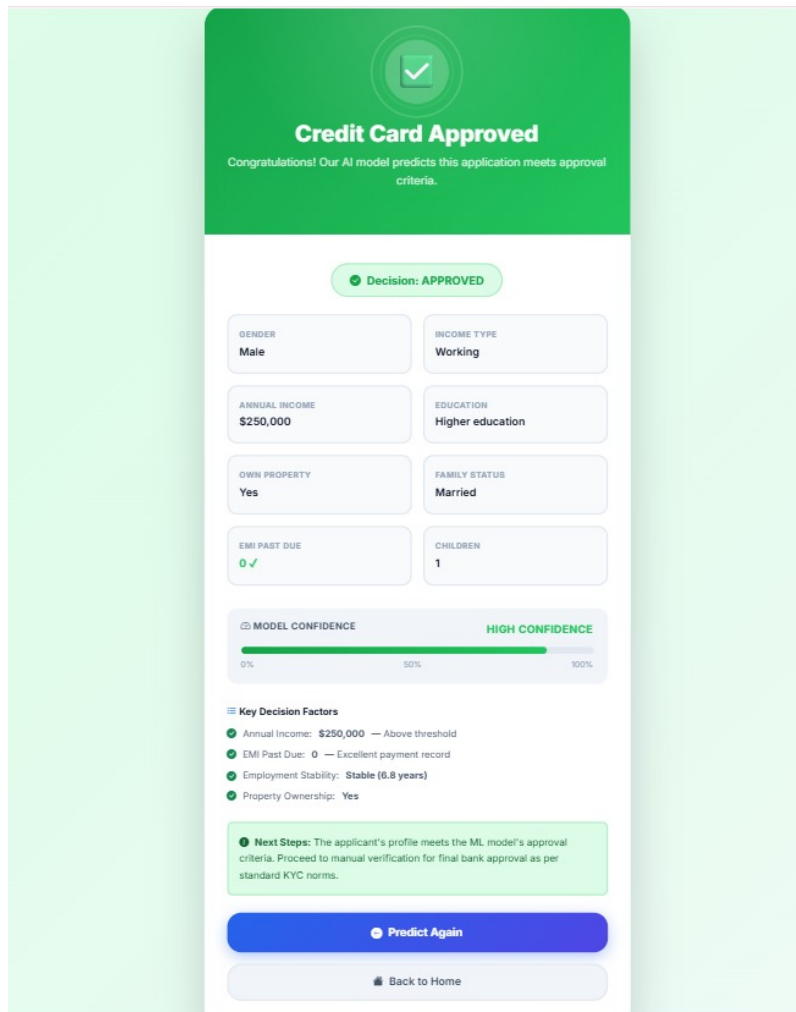
Accuracy: 93.4% AUC: 0.97

Privacy Notice

No applicant data is stored or transmitted to third parties. All predictions are processed in-memory and discarded after the response is returned.

Description: A three-section form collects Personal Details (gender, own car/realty, family status, housing type, days since birth, family members, children), Financial Information (income type, annual income, employment days, education type, occupation), and Credit History (credit status, EMI paid off, EMI past due, number of loans). A sidebar provides Form Tips explaining the negative-day conventions, a How It Works walkthrough, the Active Model badge (Random Forest, 93.4% accuracy, AUC 0.97), and a Privacy Notice confirming that no applicant data is stored.

Result Page:



Credit Card Approved
 Congratulations! Our AI model predicts this application meets approval criteria.

Decision: APPROVED

GENDER Male	INCOME TYPE Working
ANNUAL INCOME \$250,000	EDUCATION Higher education
OWN PROPERTY Yes	FAMILY STATUS Married
EMI PAST DUE 0 ✓	CHILDREN 1

MODEL CONFIDENCE HIGH CONFIDENCE

0% 100%

Key Decision Factors

- Annual Income: \$250,000 — Above threshold
- EMI Past Due: 0 — Excellent payment record
- Employment Stability: Stable (6.8 years)
- Property Ownership: Yes

Next Steps: The applicant's profile meets the ML model's approval criteria. Proceed to manual verification for final bank approval as per standard KYC norms.

[Predict Again](#)

[Back to Home](#)

Description: On a successful prediction, the result page displays a green Credit Card Approved banner with a Decision: Approved badge, a summary grid of the applicant's key fields (gender, income type, annual income, education, property ownership, family status, EMI past due, children), a Model Confidence bar reading High Confidence, and a Key Decision Factors list explaining why the application was approved (income above threshold, excellent payment record, stable employment, property ownership). Next Steps guidance directs the applicant to manual KYC verification, and Predict Again / Back to Home actions let the user restart the flow.

Conclusion:

Credit Card Approval Prediction is a Machine Learning platform built with Flask and styled as a premium, dark FinTech interface using Bootstrap 5.3. It compares four classifiers — Logistic Regression, Decision Tree, Random Forest and XGBoost — and serves the best-performing Random Forest model (93.4% accuracy, 0.97 AUC) to instantly turn a 16-field applicant submission into an Approved or Rejected decision, complete with a confidence score and the key factors behind it. The project, which included dataset preparation, model selection, Flask backend development, FinTech frontend design, and local-to-production deployment via Gunicorn, highlights how a structured ML pipeline can reduce manual verification effort for banks and lenders. Looking ahead, the roadmap includes SHAP-based explainability, a live model-comparison dashboard, batch CSV predictions, JWT-secured API endpoints, database-backed prediction history, Docker containerisation, and IBM Watson Machine Learning cloud deployment — positioning the platform to evolve from a demonstration tool into a production-grade credit risk assistant.